DATAGIA
Plataforma Digital Agroalimentaria de Castilla y León

INSTITUTO TECNOLÓGICO AGRARIO
Junta de Castilla y León
Consejería de Agricultura, Ganadería y Desarrollo Rural

Ficha técnica del modelo a33 · cereals-reuse-strategy-optimizer

Especificación técnica para integración en la plataforma DatagIA

Agosto 2026

Financiado por la Unión Europea
NextGenerationEU

Gobierno de España · Ministerio para la Transición Ecológica y el Reto Demográfico
Plan de Recuperación, Transformación y Resiliencia
Junta de Castilla y León

1. Información general del modelo

Nombre del modelo (model_key)	ml33-cereals-reuse-strategy-optimizer
Versión	2.0
Descripción	Optimizador determinista de asignación (Programación Lineal Entera Mixta, scipy.optimize.milp, backend HiGHS) que decide a qué estrategia de reutilización enviar cada lote de subproducto cerealista generado — Animal feed, Composting, Biochar o Biomass combustion — minimizando la emisión de CO2 simulada, bajo capacidades diarias de planta que se resetean cada 'lots_per_day' lotes. No es un modelo con pesos entrenados: resuelve un problema de asignación exacta por bloque a partir de una función de emisión determinista. La neuroevolución (NEAT) que trae el delivery original se mantiene por el equipo de IA únicamente como referencia comparativa (benchmark), no como motor desplegado.
Tipo de tarea (task_type)	optimization (asignación exacta MILP)
Framework	scipy.optimize.milp (HiGHS)==1.11+, numpy, pandas, pydantic
Fichero del artefacto	Sin pesos entrenados ni artefactos serializados. El 'modelo' es código determinista: la función de emisión strategy_emissions() más el solver MILP sobre bloques de lots_per_day lotes.
Objetivo del modelo	Reducir la emisión de CO2 asociada a la gestión de subproductos de cereal (cascarilla, paja, polvo de silo, salvado), decidiendo de forma óptima y auditable a qué vía de reutilización enviar cada lote generado, respetando las capacidades operativas diarias de cada vía.
Caso de uso principal	Servido en /models/ml33-cereals-reuse-strategy-optimizer/predict con dos modos por el discriminador 'mode'. Modo inline: recibe una lista de 1 o más lotes (volumen, humedad, tipo de subproducto, estación) más los parámetros de ejecución (lots_per_day, capacidades por estrategia, estrategia de guarda); devuelve la estrategia asignada por lote, su fuente (solución MILP exacta o guarda defensiva), la emisión estimada y un resumen de distribución. Modo batch: recibe la ruta de un CSV con los mismos 4 campos por fila y aplica el mismo motor sobre todo el fichero, siempre con el régimen de capacidades por defecto (las capacidades solo son configurables en modo

	inline). El reentrenamiento por usuario no aplica: /train devuelve HTTP 501 (optimizador determinista, no hay pesos que ajustar).

TABLA 1 INFORMACIÓN GENERAL DEL MODELO

2. Campos de entrada (inputs)

Nombre del campo	Tipo	Descripción	Obligatorio	Valor por defecto
generated_volume_tons	float (t)	Volumen del lote de subproducto generado	Sí	-
moisture_pct	float (%)	Humedad del lote	Sí	-
subproduct_type	str	Categoría del subproducto: Husk, Straw, Silo dust o Bran	Sí	-
season	str	Estación (Dry/Rainy). Validada como columna requerida pero NO interviene en la fórmula de emisión — campo vestigial del esquema de entrada de NEAT en el delivery original	Sí	-
process_temperature_c	float | null	Aceptado por compatibilidad de esquema; NUNCA usado como entrada de decisión — la temperatura se deriva de la estrategia elegida, no al revés	No	null
lots_per_day	int	N° de lotes antes de resetear las capacidades (bloque operativo/día simulado) [solo modo inline]	No	15
animal_feed_capacity / composting_capacity / biochar_capacity /	float (t)	Capacidad diaria de cada estrategia [solo modo inline]	No	90.0 / 140.0 / 45.0 / 10000.0

Nombre del campo	Tipo	Descripción	Obligatorio	Valor por defecto
biomass_combustion_capacity				
fallback_strategy	str	Estrategia de guarda si el MILP no puede colocar un lote (con las capacidades por defecto no llega a activarse) [solo modo inline]	No	Biomass combustion
data_path	str	Modo batch: ruta a un CSV con un lote por fila (mismas 4 columnas obligatorias)	Sí (modo batch)	-

TABLA 2 CAMPOS DE ENTRADA (INPUTS)

3. Campos de salida (outputs)

Nombre del campo	Tipo	Descripción	Rango de valores posibles	Cómo interpretar el resultado
ai_assigned_strategy	str	Estrategia de reutilización asignada al lote	Animal feed | Composting | Biochar | Biomass combustion	Es la decisión operativa: a qué vía enviar el lote
ai_assignment_source	str	Origen de la decisión	exact_min_emissions | capacity_fallback	exact_min_emissions = solución óptima probada por el MILP; capacity_fallback = guarda defensiva (no ocurre con las capacidades por defecto)
ai_is_fallback	bool	True si se usó la guarda defensiva en vez del MILP	true/false	Señal de alerta operativa si aparece con frecuencia: indica

Nombre del campo	Tipo	Descripción	Rango de valores posibles	Cómo interpretar el resultado
				capacidades insuficientes para el volumen entrante
estimated_emissions_kg	float (kg CO2)	Emisión de CO2 estimada del lote bajo la estrategia asignada	positivo	Menor es mejor; es la magnitud que minimiza el MILP
distribution / capacity_fallback_count / total_estimated_emissions_kg	dict / int / float	Agregados de la petición: conteo/% por estrategia, nº de lotes en guarda defensiva y emisión total estimada	-	Resumen de la tanda de lotes procesada, útil para un lote o para un batch completo

TABLA 3 CAMPOS DE SALIDA (OUTPUTS)

4. Métricas de evaluación

Métrica	Valor	Descripción	Interpretación (qué significa para el usuario)
Reducción de emisión vs baseline (declarada por el equipo de IA)	27.63 %	models/metrics/inference_evaluation_report.json del delivery, sobre el split de test completo (10 000 filas) con capacidades por defecto, frente a una estrategia baseline sintética	el motor exacto reduce en más de una cuarta parte la emisión simulada frente al reparto de referencia
Robustez Monte Carlo (memoria §7.2)	media 27.66 % · p05 27.27 % · p95 28.04 %	200 ejecuciones (semilla 42) muestreando la	incluso el escenario desfavorable (p05) mantiene una

Métrica	Valor	Descripción	Interpretación (qué significa para el usuario)
		temperatura de proceso por estrategia desde distribuciones normales. Métrica diagnóstica adicional frente a la línea base observada del dataset: 27.39 %	reducción superior al 27 %, muy por encima del umbral conservador del 10 % que fija la memoria
Reducción vs ladder de comparación (declarada)	exacto 27.63 % / codicioso 12.19 % / NEAT 11.31 %	models/metrics/baseline_comparison.json: el MILP exacto (desplegado) domina al heurístico codicioso por lote y al benchmark neuroevolutivo por resolver todo el bloque a la vez, no lote a lote	justifica por qué el motor desplegado es el MILP exacto y no la política aprendida
Verificación golden dataset (post-integración)	3/3 casos exactos + 10 000/10 000 filas exactas	Los 3 golden cases del manifest y, adicionalmente, el split de test completo (10 000 filas) reproducidos sin ninguna discrepancia frente a la salida auditada del delivery original — ver outputs/a33/verification_report.md	el port del motor a este repo es numéricamente idéntico al del equipo de IA, no una reimplementación aproximada

TABLA 4 MÉTRICAS DE EVALUACIÓN

5. Limitaciones conocidas

Tipo de limitación	Descripción (cuándo no usar / qué puede fallar)
Dataset sintético	Todo el dataset de referencia se generó por simulación (CTGAN + reglas causales configurables en config/data_generation_params.json), no procede de mediciones de planta real. El 27.63% de reducción es una estimación sobre un entorno de decisión sintético y requiere validación con datos reales antes de usarse como cifra operativa.
No es un modelo aprendido	Es un solver de asignación exacta; su calidad depende de la fórmula de emisión por estrategia y de las capacidades configuradas, no de un entrenamiento con datos.
'season' no afecta a la decisión	El campo season es obligatorio en el contrato de entrada pero la fórmula de emisión no lo utiliza — es vestigial del esquema de entrada del benchmark NEAT del delivery original.
Capacidades solo configurables en modo inline	El modo batch siempre usa el régimen de capacidades por defecto (90/140/45/10000 t); para simular otras capacidades hay que usar el modo inline.
Latencia en CSV grandes	El modo batch resuelve un MILP por cada bloque de forma secuencial dentro de una única petición HTTP síncrona; para el split de test completo (10 000 filas / 667 bloques) la respuesta tarda del orden de minutos en un entorno de un solo worker.
NEAT no expuesto	El delivery original incluye una política neuroevolutiva (NEAT) y su artefacto (winner_genome.pkl) como benchmark de referencia; este plugin implementa únicamente el motor MILP exacto, que la propia comparación del equipo de IA identifica como superior (27.63% vs 11.31% de NEAT).
Optimiza solo emisiones, no costes	La función objetivo minimiza exclusivamente la huella de CO2 simulada. No incluye costes de procesamiento ni de transporte por tonelada, así que una asignación óptima en emisiones puede ser económicamente inviable para la explotación (memoria §10.2). La propia memoria propone una función multiobjetivo coste-emisiones como evolución natural.
Semilla sintética que refuerza los supuestos de partida	El generador CTGAN se entrenó con solo 1.500 registros semilla construidos a partir de reglas termodinámicas y penalizaciones impuestas por los desarrolladores. No

Tipo de limitación	Descripción (cuándo no usar / qué puede fallar)
	descubre patrones del mundo real, amplifica esos supuestos, y el óptimo calculado hereda cualquier sesgo del simulador de emisiones (memoria §10.1).
Escalabilidad del solver	Cada bloque de 15 lotes se resuelve en milisegundos, pero el problema de asignación generalizada es NP-duro en general. Bloques mucho mayores (lots_per_day alto) exigirían resolver por sub-bloques o fijar un límite de tiempo al solver (memoria §10.2).

TABLA 5 LIMITACIONES CONOCIDAS

6. Observaciones técnicas

Ejecutable en CPU, sin GPU ni dependencias pesadas de ML (única dependencia nueva: scipy>=1.11.0, ya transitiva vía scikit-learn en este repo, añadida explícitamente a requirements.txt por ser ahora importada directamente). Determinista y reproducible: sin semilla aleatoria, sin artefactos que cargar. Solve MILP del orden de milisegundos por bloque de 15 lotes; un batch de 10 000 filas (667 bloques) tarda del orden de minutos en un entorno de un solo worker por ser secuencial dentro de una petición síncrona. mlflow_utils.py presente por convención del repo (siempre devuelve None: no hay artefacto de usuario que descargar). train() → HTTP 501. Sin excepción de dominio nueva: la infactibilidad se resuelve con una guarda defensiva interna (ai_assignment_source=capacity_fallback), no con un error 422.

7. Fichero de prueba de referencia

Nombre del fichero	Golden dataset del manifest (inbox/a33/manifest.yaml): 2 bloques de 15 lotes consecutivos de data/split/dataset_optimization_cereal_co2_test_raw.csv + data/predictions/inference_with_constraints.csv, y el split

	de test completo (10 000 filas) como verificación exhaustiva adicional.
Número de registros / muestras	3 golden cases (2 bloques de 15 lotes + 1 caso agregado sobre 10 000 filas), más verificación exhaustiva fila a fila sobre las 10 000 filas completas del split de test original.
Resultado esperado con este fichero	3/3 golden cases reproducidos exactos (asignación categórica sin margen de tolerancia) y 10 000/10 000 filas del split de test completo sin ninguna discrepancia frente a la salida auditada del delivery original (distribución 37.18/27.27/17.86/17.69% y emisión total 30 893 908.4 kg, ambas exactas). Ver outputs/a33/verification_report.md.

TABLA 6 FICHERO DE PRUEBA DE REFERENCIA